

Math Camp 2026

Day 3 — Probability | Visualization and Quarto

Zhilin Lu

17 August 2026

Probability

Where this is heading

Every inferential claim you make rests on a sampling distribution.

- Sample spaces and events — the set theory from Day 1, renamed
- Random variables and their distributions
- Expectation and variance
- LLN and CLT — why the normal distribution keeps appearing

Sample spaces and events

- **Sample space** Ω : every possible outcome
- **Event** $A \subseteq \Omega$: a subset of outcomes
- One die: $\Omega = \{1, 2, 3, 4, 5, 6\}$, “even” is $A = \{2, 4, 6\}$

Events combine exactly like sets:

$$A \cup B \quad A \cap B \quad A^c$$

The axioms

For any event A :

1. $P(A) \geq 0$
2. $P(\Omega) = 1$
3. If A and B are disjoint, $P(A \cup B) = P(A) + P(B)$

Everything else follows. Two consequences worth memorizing:

$$P(A^c) = 1 - P(A)$$

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

Subtract the overlap or you count it twice.

Counting

Permutations — order matters:

$$P(n, k) = \frac{n!}{(n - k)!}$$

Combinations — order does not:

$$\binom{n}{k} = \frac{n!}{k!(n - k)!}$$

“How many ways to choose 3 countries from 12?” is $\binom{12}{3} = 220$.
The binomial coefficient reappears in the binomial distribution for exactly this reason: counting which trials succeeded.

Joint, marginal, conditional

Three probabilities you can read off the same table.

- **Joint:** $P(A \cap B)$ — both happen
- **Marginal:** $P(A)$ — A happens, ignoring B
- **Conditional:** $P(A \mid B)$ — A happens, given that B did

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0$$

The names come from where the numbers sit in a cross-tabulation: joint in the cells, marginal in the margins.

A worked table

200 legislators, by party and by how they voted on a bill.

	Yes	No	Total
Government	72	48	120
Opposition	18	62	80
Total	90	110	200

Joint — the cells, divided by 200:

$$P(\text{Gov} \cap \text{Yes}) = \frac{72}{200} = 0.36$$

Marginal — the margins, divided by 200:

$$P(\text{Gov}) = \frac{120}{200} = 0.60 \quad P(\text{Yes}) = \frac{90}{200} = 0.45$$

Conditioning shrinks the sample space

$$P(\text{Yes} \mid \text{Gov}) = \frac{72}{120} = 0.60$$

Note the denominator. We no longer divide by 200 — having conditioned on Government, only those 120 legislators are still in play.

$$P(\text{Yes} \mid \text{Opp}) = \frac{18}{80} = 0.225$$

Formally the same thing, using the definition:

$$P(\text{Yes} \mid \text{Gov}) = \frac{P(\text{Yes} \cap \text{Gov})}{P(\text{Gov})} = \frac{0.36}{0.60} = 0.60$$

Conditioning is: restrict to a row, then renormalize so it sums to one.

Order matters

$$P(\text{Yes} \mid \text{Gov}) = \frac{72}{120} = 0.60 \quad P(\text{Gov} \mid \text{Yes}) = \frac{72}{90} = 0.80$$

Same cell, different denominators, different questions:

- “Of government members, what share voted yes?”
- “Of yes votes, what share came from the government?”

Confusing these two is the single most common probability error in applied work. It has a name — the prosecutor’s fallacy — and Bayes’ rule is the machinery for converting one into the other.

Multiplication and total probability

Rearranging the definition gives the **multiplication rule**:

$$P(A \cap B) = P(A \mid B) P(B)$$

Splitting an event across a partition gives the **law of total probability**:

$$P(A) = P(A \mid B)P(B) + P(A \mid B^c)P(B^c)$$

Check it on the table:

$$P(\text{Yes}) = (0.60)(0.60) + (0.225)(0.40) = 0.36 + 0.09 = 0.45 \checkmark$$

This is how the denominator of Bayes' rule is usually computed: you rarely know $P(A)$ directly, but you often know it conditional on each case.

Independence

A and B are independent when knowing one tells you nothing about the other:

$$P(A \mid B) = P(A) \iff P(A \cap B) = P(A)P(B)$$

In the table, $P(\text{Gov})P(\text{Yes}) = (0.60)(0.45) = 0.27$, but the actual joint probability is 0.36. Party and vote are **not** independent — which is the whole point of studying legislatures.

- Independence is an assumption about the world, not something you read off a table
- Disjoint is **not** independent. Disjoint events are maximally informative: if A happened, B certainly did not.
- The i.i.d. assumption does enormous work in every standard error you will report

Bayes' rule

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}$$

with the denominator usually expanded by the law of total probability:

$$P(B) = P(B \mid A)P(A) + P(B \mid A^c)P(A^c)$$

It reverses the direction of conditioning: from $P(\text{evidence} \mid \text{hypothesis})$ to $P(\text{hypothesis} \mid \text{evidence})$.

That move is behind every diagnostic-testing puzzle, and behind Bayesian inference generally.

Random variables

A **random variable** maps outcomes to numbers.

- **Discrete**: countable values — coin flips, seat counts, protest events
- **Continuous**: values in an interval — vote share, GDP, latent traits

Convention: capital X is the random variable, lowercase x a value it might take. So $P(X = x)$ reads “the probability that the random variable takes the value x .”

The point of the abstraction: once outcomes are numbers, you can average them, take their variance, and put them in a model.

PMF: the discrete case

The **probability mass function** gives the probability of each value.

$$p(x) = P(X = x), \quad p(x) \geq 0, \quad \sum_x p(x) = 1$$

Number of coalition partners a party takes on, say:

x	0	1	2	3
$p(x)$	0.4	0.3	0.2	0.1

The masses sum to 1. Each one is a genuine probability, and each can be read straight off the table.

PDF: the continuous case

For continuous X there is no mass at a point. Probability lives in **areas** under the **density**:

$$P(a \leq X \leq b) = \int_a^b f(x) dx, \quad \int_{-\infty}^{\infty} f(x) dx = 1$$

Two consequences that surprise people:

- $P(X = x) = 0$ for every single x — a point has no width, so no area
- $f(x)$ can exceed 1. A Uniform(0, 0.5) has density 2 everywhere on its support.

Density is not probability. Only integrals of density are.

Because points have zero probability, $P(X \leq x)$ and $P(X < x)$ are identical here — unlike in the discrete case.

CDF: both cases at once

$$F(x) = P(X \leq x)$$

The cumulative distribution function works for discrete and continuous variables alike, which makes it the more general object.

- Non-decreasing, from 0 at $-\infty$ to 1 at $+\infty$
- Discrete: a step function, jumping by $p(x)$ at each value
- Continuous: smooth, and $F'(x) = f(x)$ — the density is the derivative of the CDF
- Interval probabilities: $P(a < X \leq b) = F(b) - F(a)$

In R the four prefixes follow this structure exactly:

```
dnorm(x)    # density    f(x)
pnorm(x)    # CDF        F(x)
qnorm(p)    # quantile: the inverse of the CDF
rnorm(n)    # random draws
```

`qnorm(0.975)` returns 1.96. That number is the CDF run backwards, and it is where the 95% interval comes from.

Expectation

The expectation is the probability-weighted average — the long-run mean.

$$E[X] = \sum_x x p(x) \qquad E[X] = \int_{-\infty}^{\infty} x f(x) dx$$

Using the table from two slides ago:

$$E[X] = (0)(0.4) + (1)(0.3) + (2)(0.2) + (3)(0.1) = 1.0$$

Note that $E[X]$ need not be a value X can take. A distribution with mean 1.0 might never produce exactly 1, and “2.4 children per household” is not a claim about any household.

Expectation of a function

To average $g(X)$, weight $g(x)$ rather than x — you do **not** need the distribution of $g(X)$:

$$E[g(X)] = \sum_x g(x) p(x)$$

$$E[X^2] = (0)(0.4) + (1)(0.3) + (4)(0.2) + (9)(0.1) = 2.0$$

And a warning that recurs all year:

$$E[g(X)] \neq g(E[X])$$

Here $E[X^2] = 2.0$ while $(E[X])^2 = 1.0$. The gap between them is exactly the variance.

Linearity of expectation

$$E[aX + b] = aE[X] + b$$

$$E[X + Y] = E[X] + E[Y]$$

$$E\left[\sum_i X_i\right] = \sum_i E[X_i]$$

The second line holds **always** — X and Y may be dependent in any way you like.

That is unusual and extremely useful. Variance gets no such gift.

Check: $E[3X + 2] = 3(1.0) + 2 = 5$.

Variance

Variance is the expected squared distance from the mean:

$$\text{Var}(X) = E[(X - E[X])^2]$$

Almost always easier to compute in this form:

$$\text{Var}(X) = E[X^2] - (E[X])^2$$

For our table: $\text{Var}(X) = 2.0 - 1.0^2 = 1.0$, so $\text{SD}(X) = 1.0$.

Squaring is what makes deviations above and below the mean both count. The price is that variance is in **squared units** — which is why we usually report the standard deviation instead.

Variance under transformation

$$\text{Var}(aX + b) = a^2 \text{Var}(X)$$

- Adding b shifts the whole distribution and changes no spread at all
- Multiplying by a scales variance by a^2 , and the standard deviation by $|a|$

Check: $\text{Var}(3X + 2) = 9(1.0) = 9$, so $\text{SD} = 3$.

This is why rescaling a variable — dollars to thousands, proportions to percentages — changes coefficients and standard errors by exactly the same factor, leaving the t statistic alone.

Variance of a sum

$$\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y) + 2 \text{Cov}(X, Y)$$

Only when $\text{Cov}(X, Y) = 0$ does this collapse to the version you half-remember:

$$\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$$

And note:

$$\text{Var}(X - Y) = \text{Var}(X) + \text{Var}(Y) - 2 \text{Cov}(X, Y)$$

Variances **add** even when you subtract. A difference in means is noisier than either mean alone — which is why comparing two groups needs more data than describing one.

Expectation ignores dependence. Variance does not. Most algebra errors on this material live in that asymmetry.

Covariance

$$\text{Cov}(X, Y) = E[(X - E[X])(Y - E[Y])] = E[XY] - E[X]E[Y]$$

Read the sign off the definition: the product is positive when both variables are on the same side of their means, negative when they are on opposite sides.

- $\text{Cov}(X, X) = \text{Var}(X)$
- $\text{Cov}(aX, bY) = ab \text{Cov}(X, Y)$
- Units are the product of the two variables' units, which makes the magnitude nearly impossible to interpret

That last point is why we standardize.

Correlation

$$\rho = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y} \in [-1, 1]$$

Dividing by both standard deviations strips out the units. What remains is scale-free and comparable across variables.

- $\rho = \pm 1$ exactly when Y is a perfect **linear** function of X
- ρ is unchanged by any linear rescaling of either variable
- In a bivariate regression, $\hat{\beta}_1 = \rho \cdot s_y/s_x$ — the slope is the correlation, rescaled

Zero correlation is not independence

Independence $\implies \text{Cov}(X, Y) = 0$. **The converse is false.**

Take $X \in \{-3, -2, -1, 0, 1, 2, 3\}$ with equal probabilities, and $Y = X^2$:

X	-3	-2	-1	0	1	2	3
Y	9	4	1	0	1	4	9

Then $\text{Cov}(X, Y) = 0$ and $\rho = 0$ — yet Y is a **deterministic function** of X . Knowing X tells you Y exactly.

ρ measures linear association only. A correlation of zero rules out a line, and nothing else. Plot your data.

Discrete distributions

Distribution	Models	$E[X]$	$\text{Var}(X)$
Bernoulli(p)	one binary trial	p	$p(1 - p)$
Binomial(n, p)	successes in n independent trials	np	$np(1 - p)$
Poisson(λ)	counts in a fixed interval	λ	λ

Almost every discrete outcome in political science is one of these three, or built from them: whether a state ratifies, how many of 50 states ratify, how many protests occurred last month.

Bernoulli and binomial

A **Bernoulli** trial is one yes/no draw. Its variance $p(1 - p)$ is largest at $p = 0.5$ and shrinks to zero at either extreme: near-unanimous outcomes carry little information.

A **binomial** counts successes across n independent trials:

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

The three parts: $\binom{n}{k}$ counts **which** trials succeeded, p^k is the probability they did, $(1 - p)^{n-k}$ that the rest did not.

Five states each ratify with probability 0.4; the probability that exactly three do:

$$\binom{5}{3} (0.4)^3 (0.6)^2 = 10 \times 0.064 \times 0.36 = 0.2304$$

```
dbinom(3, size = 5, prob = 0.4)    # 0.2304
```

Poisson

For counts with no natural upper bound: protest events per month, coups per decade, militarized disputes per dyad-year.

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

With $\lambda = 1.5$ events per month:

```
dpois(2, lambda = 1.5)      # 0.2510  exactly two  
1 - dpois(0, lambda = 1.5) # 0.7769  at least one
```

Mean equals variance. That is a real restriction, and it is testable — when your count data has variance well above its mean (overdispersion, the usual case), this is the assumption that failed, and it is why negative binomial models exist.

Continuous distributions

Uniform(a, b) — constant density $\frac{1}{b-a}$ on $[a, b]$:

$$E[X] = \frac{a+b}{2} \quad \text{Var}(X) = \frac{(b-a)^2}{12}$$

Normal(μ, σ^2) — the one you will meet everywhere:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Symmetric, single-peaked, and completely described by two numbers: μ places it, σ^2 spreads it. You will never need to evaluate that formula by hand.

Standardizing

$$Z = \frac{X - \mu}{\sigma} \sim N(0, 1)$$

Subtract the mean, divide by the standard deviation, and any normal becomes the standard normal. Z is then “how many standard deviations from the mean” — a unit-free quantity you can compare across variables.

Within	Probability
$\mu \pm 1\sigma$	0.683
$\mu \pm 2\sigma$	0.954
$\mu \pm 3\sigma$	0.997

The 1.96 in every confidence interval is just z at 97.5%:

```
qnorm(0.975) # 1.959964
1 - pnorm(60, mean = 50, sd = 10) # 0.1587
```

Why the normal is everywhere

Not because variables in the world are normal — most are not. Income, party size and conflict counts are all badly skewed.

The normal matters because of what happens to **averages**:

Sums and means of many independent quantities tend toward a normal distribution, whatever the shape of the thing being averaged.

Your estimators — sample means, regression coefficients, differences in proportions — are all sums in disguise. That is the Central Limit Theorem, and it is the next slide.

Law of Large Numbers

As n grows, the sample mean converges to the population mean:

$$\bar{X}_n \rightarrow E[X]$$

```
set.seed(1234)
draws <- rbinom(10000, size = 1, prob = 0.3)
plot(cumsum(draws) / seq_along(draws), type = "l", ylim =
abline(h = 0.3, col = "red"))
```

The running average wanders early and settles down.

The LLN says the estimate ends up in the right place. It says nothing about how fast, and nothing about the shape of the error along the way. Both of those come from the CLT.

Central Limit Theorem

For large n , whatever the shape of the underlying distribution:

$$\bar{X}_n \approx N\left(\mu, \frac{\sigma^2}{n}\right)$$

Three claims packed into one line:

1. The sampling distribution of the mean is approximately **normal**
2. It is centred on the true μ
3. Its spread is $\sigma/\sqrt{n}$ — shrinking, but only with the **square root** of n

Point 1 is the surprising one. It holds regardless of what you sampled from.

CLT by simulation

The exponential distribution is heavily right-skewed — nothing like a bell curve.

```
set.seed(1234)
means <- replicate(5000, mean(rexp(50, rate = 1)))
hist(means, breaks = 40)
sd(means)           # 0.139
```

For Exponential(1), $\mu = 1$ and $\sigma = 1$, so theory predicts a standard deviation of $1/\sqrt{50} = 0.141$. The simulation gives 0.139.

Draw from a skewed distribution, average 50 of them, repeat 5000 times: **the histogram of those means is symmetric and bell-shaped.**

What the CLT does not say

- **Not** that your data are normal. It is a statement about the *sampling distribution of a statistic*, not about X .
- **Not** that $n = 30$ suffices. That rule of thumb fails badly for heavily skewed data or rare binary outcomes.
- **Not** that outliers stop mattering. A distribution with infinite variance has no CLT at all.
- **Not** a licence to ignore dependence. Clustered or serially correlated data breaks the i.i.d. assumption, and the usual standard errors are then wrong.

The CLT is what makes normal-based inference legitimate on non-normal data. It is not a guarantee that inference is safe.

Sampling distributions

Three distributions that are easy to conflate:

1. **Population** — how X is distributed in the world
2. **Sample** — how your n observed values are distributed
3. **Sampling distribution** — how a *statistic* would be distributed across hypothetical repeated samples

Only the second one is ever observed. The first is what you want to know; the third is what makes the inference possible.

Standard error

The standard error is the standard deviation of that third distribution:

$$SE(\bar{X}) = \frac{\sigma}{\sqrt{n}}$$

n	SE (with $\sigma = 1$)
100	0.100
400	0.050
1600	0.025

Quadrupling the sample halves the standard error. That single fact is the arithmetic behind every power calculation, and behind why the marginal survey respondent is worth so much less than the first one.

Every “ $\pm$ ” you report for the rest of your career is a claim about this distribution. `## Morning wrap-up {.shrink}`

You should now be able to:

- Read joint, marginal and conditional probabilities off a cross-tabulation, and keep $P(A | B)$ apart from $P(B | A)$
- Distinguish a PMF from a PDF, and density from probability
- Compute $E[X]$ and $\text{Var}(X)$ from a table, and use their properties — including where independence is required and where it is not
- Say why zero correlation does not mean independence
- Name what the binomial, Poisson and normal are for
- State what the CLT claims and what it does not
- Say precisely what a standard error is the standard deviation of

Visualization and Quarto

Two tools, one goal

A figure and a document that regenerate themselves when the data changes.

- ggplot2 for graphics built from a consistent grammar
- Quarto for documents where the numbers in the text come from the code

Everything you produce this year — problem sets, memos, the job market paper — should be built this way.

We start from the file you saved yesterday afternoon.

```
library(tidyverse)
analysis <- readRDS("data/clean/analysis.rds")
glimpse(analysis)
```

Grammar of graphics

ggplot2 rests on one idea: **every statistical graphic is the same structure with different parts filled in.**

a plot = data + mapping + geometry

Piece	The question it answers
data	which table
aesthetics (aes)	which column goes on which visual channel
geoms	what shape to draw
scales	how values become positions and colours
facets	how to split into panels
labs, themes	everything that is not data

You are not memorising forty kinds of plot. You are learning one structure.

Data and mapping, no geometry

```
ggplot(analysis, aes(x = gdp_pc, y = vote_share))
```

An empty coordinate system.

The data is attached, the axes know what they represent, the ranges have already been computed — nothing has been said about **what shape to draw**.

That missing piece is the geometry.

Add a geometry

```
ggplot(analysis, aes(x = gdp_pc, y = vote_share)) +  
  geom_point()
```

The points appear.

Now swap `geom_point()` for `geom_line()`, `geom_boxplot()`, `geom_hex()`. **Nothing before the + changes.** Different plot, same first line.

That is the payoff of the grammar: new kinds of figure cost you one word, not a new function with new arguments.

Why + and not |>

You will ask, so: **historical accident**. ggplot2 predates the pipe, and it uses + to mean “add another layer”.

Inside ggplot, +. Outside ggplot, |>.

There is no deeper principle. Do not go looking for one.

Overplotting

312 points on top of each other hide the shape of the data.

```
ggplot(analysis, aes(x = gdp_pc, y = vote_share)) +  
  geom_point(alpha = 0.4)
```

alpha sets opacity between 0 and 1. Where points pile up the colour deepens, so density becomes visible.

For heavier overplotting: `geom_hex()`, `geom_bin2d()`, or `geom_jitter()` when a variable is discrete but plotted as continuous.

Log scales

```
ggplot(analysis, aes(x = gdp_pc, y = vote_share)) +  
  geom_point(alpha = 0.4) +  
  scale_x_log10()
```

Income is right-skewed: a few rich countries stretch the axis and squash everyone else into the left margin.

`scale_x_log10()` fixes the display without touching the data. Alpha plus a log axis is close to standard practice for income, population, GDP, trade volumes — anything that spans orders of magnitude.

aes(): the error

```
ggplot(analysis, aes(x = gdp_pc, y = vote_share)) +  
  geom_point(color = "region")
```

Error: Invalid colour: region

geom_point() took "region" as the literal name of a colour, and there is no colour called region.

This one at least stops you.

`aes()`: the version that does not stop you

```
ggplot(analysis, aes(x = gdp_pc, y = vote_share, color = "steelblue"))  
  geom_point()
```

A plot appears. The points are **red**, and there is a legend labelled "steelblue" with one entry.

ggplot read "steelblue" as a *variable* that takes the same value for all 312 rows, built a one-level legend for it, and assigned it the first colour in the palette.

No error, no warning, wrong figure. This is the failure mode worth fearing.

aes(): both correct versions

Colour **varies with the data**, so it belongs inside aes():

```
ggplot(analysis, aes(x = gdp_pc, y = vote_share, color = r  
  geom_point(alpha = 0.6) +  
  scale_x_log10()
```

Colour is **fixed**, so it belongs outside:

```
ggplot(analysis, aes(x = gdp_pc, y = vote_share)) +  
  geom_point(alpha = 0.6, color = "steelblue") +  
  scale_x_log10()
```

The rule *Varies with the data* → *inside aes()*. *Constant* → *outside*.

The same rule governs size, shape, alpha, fill, and linewidth.

Two diagnostics for when a figure looks wrong:

- A legend you did not want → something constant ended up inside `aes()`
- Everything one colour → a variable ended up outside it

One variable: histogram

```
ggplot(analysis, aes(x = vote_share)) +  
  geom_histogram(bins = 30)
```

Only x is mapped. The counts on the vertical axis are computed by ggplot.

Always set bins yourself. Leave it out and ggplot uses 30 and tells you so — the message is there because the right number depends on your data, and the choice materially changes what you see. Try 10, 30, 60 and watch the shape move.

One variable: density

```
ggplot(analysis, aes(x = vote_share)) +  
  geom_density()
```

A smoothed version, with no bins to choose.

But there is still a choice: the smoothing **bandwidth**, set by `adjust`. A histogram shows its arbitrariness on the face of the plot; a density curve hides it.

Drawing both is a cheap habit and occasionally an informative one.

Comparing groups: boxplot

```
ggplot(analysis, aes(x = region, y = vote_share)) +  
  geom_boxplot()
```

Box spans the interquartile range, the line is the median, whiskers reach $1.5 \times \text{IQR}$, anything beyond is drawn individually.

A boxplot hides multimodality by construction — two clusters and one broad blob can produce the same box. `geom_violin()` or `geom_jitter()` on top will show you.

Ordering categories

```
ggplot(analysis, aes(x = fct_reorder(region, vote_share),  
                     y = vote_share)) +  
  geom_boxplot() +  
  labs(x = "Region")
```

Alphabetical order carries no information about your data.

`fct_reorder()` sorts by the median of the second variable.

It is from **forcats**, which loads with the tidyverse, and it is the single function from that package you will use most.

Lines: the group trap

```
ggplot(analysis, aes(x = year, y = gdp_pc)) +  
  geom_line(alpha = 0.4)
```

One jagged line zig-zagging across the whole panel.

ggplot has no idea which points belong to the same country, so it connected all 312 of them in order of the x axis.

This is the most common mistake in plotting panel data.

Lines: fixed

```
ggplot(analysis, aes(x = year, y = gdp_pc, group = country))  
  geom_line(alpha = 0.4)
```

Twelve separate lines.

group tells ggplot which rows form one line. Mapping color = country would group them implicitly too — but then you also get a twelve-colour legend, which is rarely what you want.

Faceting

```
ggplot(analysis, aes(x = year, y = gdp_pc, group = country))  
  geom_line(alpha = 0.5) +  
  facet_wrap(~ region)
```

Small multiples: one panel per level, same axes throughout.

The tilde reads as “by”. `facet_wrap(~ var)` wraps panels into a block; `facet_grid(rows ~ cols)` crosses two variables fully.

Free scales

```
facet_wrap(~ region, scales = "free_y")
```

A trade-off with no correct answer:

	Shared axes	Free axes
Good for	comparing levels across panels	seeing shape within a panel
Risk	small panels flatten to nothing	panels look alike when they are not

Choose according to what you want the reader to see, and say which you chose.

Facets or colours

The instinct is to put five groups in one panel with five colours. It rarely reads well.

Past three or four colours, facet instead.

Colour is best spent on one variable that genuinely matters, or on highlighting one series against grey.

Adding a fitted line

```
ggplot(analysis, aes(x = gdp_pc, y = vote_share)) +  
  geom_point(alpha = 0.3) +  
  geom_smooth(method = "lm") +  
  scale_x_log10()
```

Layers are drawn in the order you add them, so the line added second sits on top of the points. Reverse them and the points hide the line.

`geom_smooth()` defaults to a loess curve; `method = "lm"` makes it linear. The band is a confidence interval — `se = FALSE` removes it.

Piping into ggplot

```
analysis |>  
  group_by(region, year) |>  
  summarize(mean_gdp_pc = mean(gdp_pc, na.rm = TRUE),  
            .groups = "drop") |>  
  ggplot(aes(x = year, y = mean_gdp_pc, color = region)) +  
  geom_line(linewidth = 0.9)
```

Yesterday's verbs and today's layers compose directly: `|>` to prepare the data, then `+` to build the figure.

Watch the switch at the `ggplot()` line. That is where the pipe stops and the plus begins.

Bars

```
analysis |>  
  filter(year == 2020) |>  
  slice_max(gdp_pc, n = 8) |>  
  ggplot(aes(x = fct_reorder(country, gdp_pc), y = gdp_pc))  
  geom_col(fill = "grey30") +  
  coord_flip() +  
  labs(x = NULL, y = "GDP per capita (USD)")
```

- `geom_col()` draws the values you supply; `geom_bar()` counts rows for you
- `coord_flip()` turns the bars sideways, which is how long category labels stay legible
- `x = NULL` drops an axis title that would only repeat the labels

Labels

```
labs(  
  title      = "Income and incumbent support",  
  subtitle   = "12 countries, 1995-2020",  
  x          = "GDP per capita, USD (log scale)",  
  y          = "Incumbent vote share (%)",  
  caption    = "Source: simulated data for teaching"  
)
```

Axis labels with units, every time.

Figures get pulled out of your paper and into slides, referee reports and other people's talks. `gdp_pc` means something to you and nothing to anyone else.

Themes

```
theme_minimal(base_size = 12)  # clean, good for papers
theme_bw()                      # framed
theme_classic()                 # axes only, common in journals
```

`base_size` scales every piece of text at once. It is the argument to reach for first: a figure for a projector needs larger type than the same figure in a manuscript.

Fine control lives in `theme()` — `legend.position = "bottom"`, `panel.grid.minor = element_blank()`, and so on.

Saving

```
p <- ggplot(analysis, aes(gdp_pc, vote_share)) +  
  geom_point(alpha = 0.35) +  
  labs(x = "GDP per capita, USD", y = "Vote share (%)") +  
  theme_minimal(base_size = 12)  
  
ggsave("output/fig1.pdf", p, width = 6.5, height = 4.5)  
ggsave("output/fig1.png", p, width = 6.5, height = 4.5, dpi = 300)
```

Format	Use for	Why
.pdf	papers, LaTeX	vector: sharp at any size
.png	slides, email	raster: set dpi = 300

Storing the plot in `p` lets you save it, keep adding layers, or reuse it later.

The ggsave() surprise

Text size in the saved file is **absolute**, not relative to the figure.

So the fix for “the labels came out too small” is to make width and height **smaller**, not larger. A bigger canvas with the same 12-point type gives you smaller-looking text.

Never save a figure with the Export button or a screenshot.

ggsave() in the script means the figure rebuilds itself when the data changes.

Colour that everyone can read

```
ggplot(analysis, aes(year, gdp_pc, color = region, group =  
  geom_line(alpha = 0.7) +  
  scale_color_viridis_d() +  
  theme_minimal()
```

Roughly 8% of men have some form of colour vision deficiency. Default red–green palettes are a real accessibility problem in published work.

`scale_color_viridis_d()` for discrete variables, `_c()` for continuous. It also survives being printed in black and white.

ggplot errors, translated

Message	What it means
object 'gdp_pc' not found	you used a column name outside aes()
Invalid colour: region	a variable name sits outside aes()
stat_bin() using bins = 30	not an error; set bins yourself
Removed 20 rows containing missing values a legend you never asked for	not an error; NAs were dropped from the plot something constant went inside aes()

The last two are worth pausing on. Both are ggplot telling you something true about your data.

Quarto

The whole idea

The numbers in your prose are computed, not typed.

That is the entire point. Everything else this hour is mechanics.

Why it matters

It is 2 a.m. You find one miscoded row. You fix the data and re-run the model.

Your draft now contains seventeen numbers that are wrong: in the abstract, in the text, in two table notes, in a footnote. You have to find every one by hand.

You will miss one.

With Quarto, those seventeen numbers update themselves.

Anatomy of a .qmd

Three ingredients in one plain-text file:

title: "Problem Set 1"

author: "Your Name"

format: html

A heading

Prose in **markdown**, with *emphasis* and ``code``.

```{r}

#| label: fig-scatter

#| echo: false

#| fig-cap: "Income and incumbent support"

ggplot(analysis, aes(gdp\_pc, vote\_share)) + geom\_point()

```

More prose after the chunk.

The YAML header

```
---  
title: "Problem Set 1"  
author: "Your Name"  
date: today  
format:  
  html:  
    toc: true  
    embed-resources: true  
---
```

Between the two --- lines, and **indentation is meaningful** — two spaces, never tabs.

format: decides the output. Change that one line and the same source becomes HTML, PDF, or Word.

Markdown, the parts you need

Syntax	Result
## Heading	section heading
bold	bold
<i>*italic*</i>	<i>italic</i>
<code>`code`</code>	monospace
- item	bullet
1. item	numbered
[text](url)	link
> quote	block quote

Blank lines separate paragraphs. Without one, two lines run together — the most frequent markdown complaint, and the easiest to fix.

Chunk options

Written with #| on the first lines inside the chunk:

Option	Effect
echo: false	run it, hide the code
eval: false	show the code, do not run it
include: false	run it, show nothing at all
warning: false	hide warnings
message: false	hide messages
fig-cap: "..."	caption the figure
fig-width, fig-height	size in inches
label: fig-xxx	name it, for cross-references

Rule of thumb: for a problem set, echo: false on figures, echo: true where the code **is** the answer.

The setup chunk

Every document starts with one:

```
```${r}  
#| include: false
library(tidyverse)
knitr::opts_chunk$set(warning = FALSE, message = FALSE)
analysis <- readRDS("data/clean/analysis.rds")
```
```

`include: false` runs it and shows nothing.

`opts_chunk$set()` sets defaults for every chunk below, so you do not repeat yourself.

Inline code

Backtick, r, expression, backtick — inside ordinary prose:

```
The sample contains `r nrow(analysis)` country-  
year observations
```

```
covering `r n_distinct(analysis$country)` countries. Mean in-  
vote share is `r round(mean(analysis$vote_share, na.rm = TRUE), 1)`
```

Renders as:

*The sample contains **312** country-year observations cover-
ing **12** countries. Mean incumbent vote share is **38.2**%.*

Not one of those three numbers is typed in the source.

Inline code on model results

```
`r`  
m <- lm(vote_share ~ log(gdp_pc) + factor(region), data = ana  
`r`
```

The coefficient on log income is `r round(coef(m)[2], 2)`
(SE `r round(sqrt(diag(vcov(m)))[2], 2)`), on `r nobs(m)` ob
*The coefficient on log income is **4.85** (SE **0.21**), on **292**
observations.*

Change the specification in the chunk and the sentence follows.

Note `nobs(m)` is **292**, not 312: rows with missing values were dropped from the regression. Typed by hand, that number would almost certainly have been wrong.

LaTeX math

| Syntax | Result |
|----------------------------|--------------------------|
| <code>\$\beta\$</code> | β inline |
| <code>\$\$... \$\$</code> | display, on its own line |

\$\$

```
\text{vote share}_{it} = \beta_0 + \beta_1 \ln(\text{GDP p.c.}_{it}) + \gamma_i + \delta_t + \varepsilon_{it}
```

\$\$

$$\text{vote share}_{it} = \beta_0 + \beta_1 \ln(\text{GDP p.c.}_{it}) + \gamma_i + \delta_t + \varepsilon_{it}$$

This is the notation you have been reading for three mornings.
You already know how to read it; now you are typing it.

Math you will actually use

| Input | Output |
|--|-----------------------------------|
| <code>\alpha \beta \sigma</code> | $\alpha \beta \sigma \varepsilon$ |
| <code>\varepsilon</code> | |
| <code>\hat{\beta} \bar{x} \tilde{y}</code> | $\hat{\beta} \bar{x} \tilde{y}$ |
| <code>\tilde{y}</code> | |
| <code>x_i x^2 x_{it}</code> | $x_i x^2 x_{it}$ |
| <code>\sum_{i=1}^n</code> | $\sum_{i=1}^n$ |
| <code>\frac{a}{b}</code> | $\frac{a}{b}$ |
| <code>\times \leq \neq \approx \sim</code> | $\times \leq \neq \approx \sim$ |
| <code>\sim</code> | |
| <code>\mathbb{E} \mathrm{Var}</code> | $\mathbb{E} \mathrm{Var}$ |

Braces for anything longer than one character. `x_{it}` is right; `x_it` renders as x_it .

Cross-references

Label a chunk `fig-something`, give it a caption, then refer to it:

As `@fig-scatter` shows, the relationship is positive.

Quarto inserts the number and links it. Add or move figures and everything renumbers.

Prefixes matter: `fig-` for figures, `tbl-` for tables, `eq-` for equations. A label without the right prefix will not resolve.

Rendering

Press **Render** in Positron, or:

```
quarto render report.qmd  
quarto render report.qmd --to html  
quarto render report.qmd --to pdf
```

Two steps happen, and knowing which one failed tells you how to fix it:

```
report.qmd  
  | 1. knitr runs every chunk, inserts the results  
plain markdown  
  | 2. pandoc converts to the output format  
report.html / report.pdf
```

Step 1 fails → a code error, and the message names the chunk.

Step 2 fails → almost always LaTeX.

Rendering uses a clean session

This is the thing to remember.

Rendering starts a **brand new R session**. Nothing you created by hand in the console exists there.

```
Error: object 'analysis' not found
```

means the line that loads the data lives only in your console, not in the document. Move it into the setup chunk.

Test it deliberately: `rm(list = ls())`, then render. If it still works, the document is self-contained.

The strictness is the feature

That clean session is not Quarto being difficult.

If it renders, someone else can run it and get exactly your output.

That is what reproducible means, and Quarto checks it for you every single time.

The same test applies to a plain script: restart R, source it from the top. If it fails, something happened by hand and was never written down.

HTML or PDF

| | HTML | PDF |
|-------------------|------------------------------|----------------------|
| Extra software | none | LaTeX |
| Speed | fast | slower |
| Use for | drafts, anything interactive | submission, printing |
| Chance of failing | near zero | high the first time |

Work in HTML. Switch to PDF once the content is finished.

Two things that will bite you

Images vanish when you email the HTML. By default figures sit in a separate `_files/` folder. Fix it in the YAML:

```
format:
  html:
    embed-resources: true
```

PDF needs LaTeX. Install it once, at home, on good wifi:

```
install.packages("tinytex")
tinytex::install_tinytex()
```

It is a few hundred megabytes. Twenty people doing this at once will take down the classroom network.

When rendering fails

| Message | Cause |
|---------------------------------|---|
| object 'x' not found | not defined in the document — clean session |
| there is no package called ... | not installed, or installed for another R |
| LaTeX Error: File ... not found | missing LaTeX package; tinytex usually self-heals |
| ! Undefined control sequence | a mistyped LaTeX command in your math |
| chunk label duplicated | two chunks share a label: |

The error message names the chunk and the line numbers. Go there and run that chunk on its own.